2023年国赛赛题F题

1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

使用提示：最好使用电池供电，让舵机处于正常工作，如果使用数据线供电，虽然舵机也能运动，但舵机电压无法达到7.4v的正常工作，舵机运动效果会差一些。

2.器材清单

- k230 x2
- 24路舵机驱动板 x2
- 二维电动云台 x2
- 绿色激光笔 x1 购买
- 7.4v电池 x1 购买
- 红色激光笔 x1 购买
- 红色激光笔支架 x1 购买
- 公对公杜邦线 x2
- 铝合金板 x2 购买

  红色激光笔需要被k230识别，客户可以自行选择合格赛题的激光笔，也可以选择器材清单上的红色激光笔作为能被识别且符合赛题的红色激光笔，固定激光笔的话，可以自己寻找合适的激光笔支架也可以选择清单上的支架选择方形支架22.5mm的款式，特别注意的是如果使用这个方形支架则需要拆下来使用其中的部分，用法见摆放位置的图片。

3.摆放位置

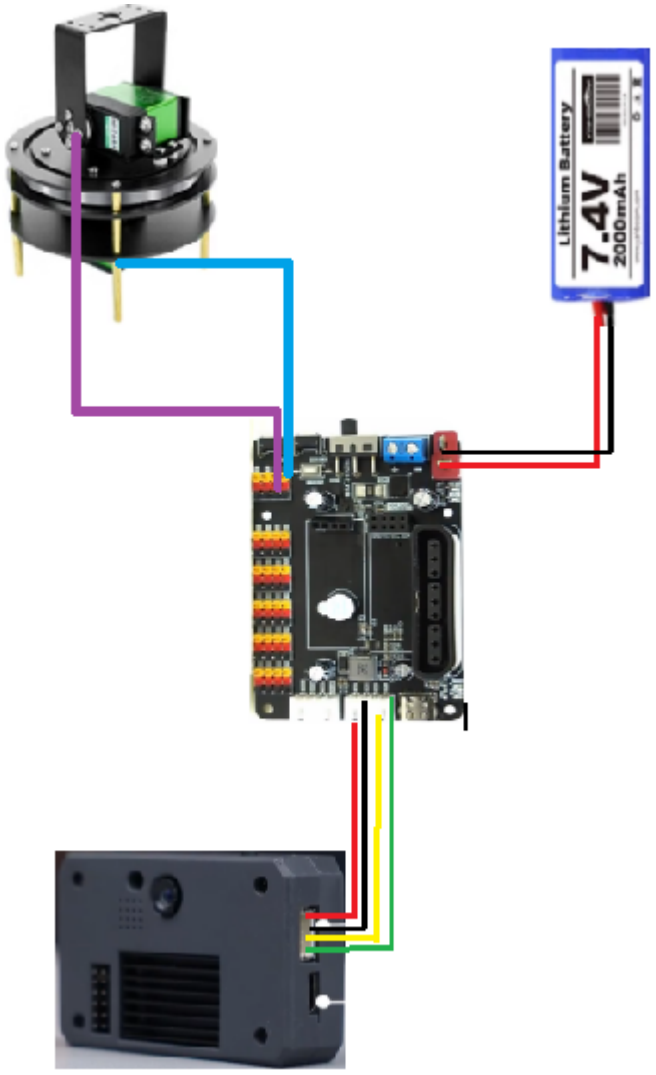


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行固定，k230与云台的距离大约7cm，二维云台系统离黑框为题目要求的1m距离。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230关键代码解析

- get_square.py

```
    # 找内框
    if flag_find_first_rect == False:
        # threshold 参数用于设置检测阈值(threshold大小和要检测矩形的像素大小有关，将阈值设定在要检测矩形的面积范围内可以减少外界其他噪声)
        # x_gradient 和 y_gradient 参数用于设置边缘梯度阈值（x_gradient 和 y_gradient越大识别越准确但耗费时间越长，x_gradient 和 y_gradient越小越快但干扰性大）
        # 两者都和环境亮度有关
        # 找矩形调整threshold，x_gradient，y_gradient这三个值
        for rect in img.find_rects(threshold = 80000, x_gradient=10, y_gradient=10):
            if rect:
                flag_find_first_rect = True
                # 获取矩形的四个角的坐标
                first_rect_corners = rect.corners()
                area = rect.magnitude() #矩形像素面积大小
                print(area)
    # 找外框
    if flag_find_first_rect == True:
        if flag_find_second_rect == False:
            for rect in img.find_rects(threshold = 60000, x_gradient=10, y_gradient=10):
                if rect:
                    area = rect.magnitude()
                    if area < 90000:
                        flag_find_second_rect = True
                        # 获取矩形的四个角的坐标
                        second_rect_corners = rect.corners()
                    print(area)
```

此代码是为了获取矩形的四个点坐标，通过找黑框的外矩形和内矩形然后确定黑框中心矩形，依次来获得中心矩形的四个顶点坐标。

- red_light.py

```
def color_blob(img,hano,color):#寻找指定颜色最大色块
    blobs = img.find_blobs([hano],roi=light_roi,merge = True)
    if blobs:
        global last_cx
        global last_cy
        for blob in blobs:
            roi = (blob.x()-5,blob.y()-5,blob.w()+10,blob.h()+10)#得到亮斑外接圆范围
            if img.find_blobs([color],roi=roi,x_stride=1, y_stride=1,
                              area_threshold=0, pixels_threshold=0,merge=False,margin=1):#找到了指定颜色
                cx = blob[5]
                cy = blob[6]
```

```
                img.draw_rectangle(blob[0:4],color=(0,0,255)) # 画矩形
                img.draw_cross(blob[5], blob[6],color=(0,0,255)) # 画十字
                last_cx = cx#更新坐标
                last_cy = cy#更新坐标
                return cx, cy, 1
        return last_cx, last_cy, 0
    return last_cx, last_cy, 0
```

此代码是为了获取目标色块的坐标，通过调用k230的找色块的函数，找到阈值内的色块，对于本题目来说就是红色色块，所以需要定义阈值为红色。

- 分割坐标函数

```
def conversion_postion(x,y):
    if x > 510:
       x_h = 255
       x_l = 255
       x_z = x-255-255
    elif 510>=x>=255:
       x_h = 255
       x_l = x-255
       x_z = 0
    else:
       x_h = x
       x_l = 0
       x_z = 0
    if y > 510:
       y_h = 255
       y_l = 255
       y_z = y-255-255
    elif 510>=y>=255:
       y_h = 255
       y_l = y-255
       y_z = 0
    else:
       y_h = y
       y_l = 0
       y_z = 0
    return x_h,x_l,x_z,y_h,y_l,y_z
```

因为给stm32只是传输8位2进制数，所以最高传输数据为255，而我们K230的采用的是640x480因此需要对其坐标进行分割，将640拆成3帧数据进行传输。

6.一号云台关键代码解析

- control.c

```
void go_to_point(uint16_t now_x, uint16_t now_y, uint16_t targ_x, uint16_t targ_y)
{
    uint16_t i;
```

```
    float step_num = 1000;/* 1000步*/
    float step_x, step_y; /* 步进值  */
    float going_x = now_x, going_y = now_y; /* 当前目标值*/

    step_x = (targ_x - now_x) / step_num;
    step_y = (targ_y - now_y) / step_num;

    for(i = 0; i < step_num; i++)
{

    kfp_x.source = step_x;
    kfp_y.source = step_y;
    kfp_x.out = KalmanFilter(&kfp_x, kfp_x.source);
    kfp_y.out = KalmanFilter(&kfp_y, kfp_y.source);
    going_x += kfp_x.out;
        going_y += kfp_y.out;

        set_target(going_x, going_y);


    }
}
```

此代码是将得到的两点坐标进行细分，从而是将矩形的2个点分成许多个点，然后许多点之间做pid运算，使得舵机运动更加精准，同时也能控制舵机在黑框上运动一周的速度。

```
uint8_t get_point1_p()
{
    uint8_t lable, load_1, load_2,load_3, ret = 0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2, &load_3) == 0){

          if(lable == 0x10){
            rec_step_point[3][0] = (load_1 + load_2 + load_3);
        } else if(lable == 0x11){
            rec_step_point[3][1] = (load_1 + load_2 + load_3);
        } else if(lable == 0x12){
            rec_step_point[0][0] = (load_1 + load_2 + load_3);
        } else if(lable == 0x13){
            rec_step_point[0][1] = (load_1 + load_2 + load_3);
        } else if(lable == 0x14){
            rec_step_point[1][0] = (load_1 + load_2 + load_3);
        } else if(lable == 0x15){
            rec_step_point[1][1] = (load_1 + load_2 + load_3);
        } else if(lable == 0x16){
            rec_step_point[2][0] = (load_1 + load_2 + load_3);
        } else if(lable == 0x17){
            rec_step_point[2][1] = (load_1 + load_2 + load_3);
            mode_flag = 4;
        }

    }
```

```
    return ret;
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起形成正确的矩形顶点坐标，然后将矩形的x坐标，y坐标依次赋给二维数组，当最后一个坐标获取完成后给定义的标志位赋值。

```
static uint8_t step_flag =0;
    if(step_flag ==0)
    {

    USART_Cmd(USART3, ENABLE);
        get_point1_p();//获得坐标

    }

    if(mode_flag == 4&&step_flag==0){
    USART_Cmd(USART3, DISABLE);//开启串口3
    TIM_ITConfig(TIM1,TIM_IT_Update, ENABLE);//开启定时器1°

        TIM_Cmd(TIM4, ENABLE); // 开启pid的定时器

        go_to_point(rec_step_point[0][0], rec_step_point[0][1], rec_step_point[1][0],
rec_step_point[1][1]);
        go_to_point(rec_step_point[1][0], rec_step_point[1][1], rec_step_point[2][0],
rec_step_point[2][1]);
        go_to_point(rec_step_point[2][0], rec_step_point[2][1], rec_step_point[3][0],
rec_step_point[3][1]);
        go_to_point(rec_step_point[3][0], rec_step_point[3][1], rec_step_point[0][0],
rec_step_point[0][1]);

    go_to_point(rec_step_point[0][0], rec_step_point[0][1], rec_step_point[1][0],
rec_step_point[1][1]);
        go_to_point(rec_step_point[1][0], rec_step_point[1][1], rec_step_point[2][0],
rec_step_point[2][1]);
        go_to_point(rec_step_point[2][0], rec_step_point[2][1], rec_step_point[3][0],
rec_step_point[3][1]);
        go_to_point(rec_step_point[3][0], rec_step_point[3][1], rec_step_point[0][0],
rec_step_point[0][1]);
    step_flag+=1;



    }
```

注意:一号云台一定要先去串口定义的函数的那个位置关闭串口3使能。对于这个代码首先定义标志位，然后打开串口3运行获取坐标函数，获取完成后会给另一个标志位赋值，然后2个标志位判断下继续执行，然后关闭串口3以免串口收数据抢占pid运算和舵机pwm运动的进程，开启pid运算定时器和pwm运动定时器，然后执行细分pid运动函数，控制舵机在矩形框内运动，运动完成后给标志位加1从而让舵机停下来。

7.二号云台关键代码解析

- control.c

```
uint8_t get_point2_p()   //处理坐标函数
{
    uint8_t lable, load_1, load_2 ,load_3 ,ret=0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2,&load_3) == 0){
     if(lable == 0x00){
            rec_step_p[0][0] = load_1 + load_2 + load_3 ;
//          ret=1;

        } else if(lable == 0xff){

            rec_step_p[0][1] = load_1 + load_2 + load_3 ;
//          ret=1;

        }

    return ret;
}
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起这样形成正确的坐标，将坐标赋值给二维数组。

```
int a_Err=(int)x_pid.Err, b_Err=(int)y_pid.Err;

    get_point2_p();
    if(abs(a_Err) + abs(b_Err) < 800){/* 声光提醒*/
          BEEP_ON;
           }
    else {
          BEEP_OFF;
           }
    set_target(rec_step_p[0][0],rec_step_p[0][1]);
```

此代码是在上面获得红色激光坐标的基础上将坐标作为pid目标值，然后控制舵机运动，同时在小于x轴y轴误差之和时发出声光提醒。

8.二维云台重要代码解析

- bsp_servo.c

```
ServTypdef Xserv = {
    .pwm = 1022,
    .MAXPWM = 2400,
    .MINPWM = 500,
    .MIDPWM = 1097
};

ServTypdef Yserv = {
    .pwm = 1022,
    .MAXPWM = 2100,
    .MINPWM = 400,
    .MIDPWM = 933
};
```

此代码是设置舵机启动的起始位置，一号云台和二号云台位置不同，所以需要修改这个来拟合云台运动的顶点，由于k230上x和y的0点在左上角，所以我将云台开机初始位置设置在左上角，然后云台可以通过k230的坐标变化来按比例进行调整，如果在调整云台位置和细调位置的时候就需要调这个位置参数了。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }

    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                         //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
```

```
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- pid.c

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
             + PID->Kd * (PID->Err - PID->LastErr);                     /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出。

- bsp_usart.c

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {            //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){      //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
```

```
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){      //帧尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

9效果展示

1.画铅笔框效果展示

该题目要求30s内在在周围运动一圈不脱离线，我们运动一圈的时间小于30s，完美达到要求且能回到中心点。

2.基础部分画框效果展示

这个题目要求在矩形框内运动一圈且时间在30s内，从视频看到时间完美符合，运动情况也比较良好，由于画正矩形过于简单，所以直接展示画任意框。

3.发挥部分云台追踪效果展示

这个题目要求红色激光笔运动一圈，绿色激光笔跟随运动二者之间不要超过3cm，在范围内的时候有声音提醒，从视频中可以看出红色激光笔运动状态良好且绿色激光笔没有超过3cm的情况，且在追到的时候有声音提醒，完美达到题目要求。